

Real-world (one reusable session) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real API client wraps a session). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

Instead of calling `requests.get(url, headers={...})` over and over and retyping the same headers dict each time, `requests` gives you a `Session` object you set headers on once — every call made *through* that session automatically carries them.

Think of a session like a browser tab that stays logged in — you sign in once, and every page you visit after that already knows who you are.

Things to use:

- `session = requests.Session()` — make one, once.
- `session.headers.update({"Authorization": "Bearer fake-token"})` — set the header once.
- `session.get(url)` — call it like `requests.get`, but through the session.
- Make 2-3 calls, then print `session.headers` to confirm the header is really attached.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

A `requests.Session()` object holds state (headers, cookies, and connection pooling) across multiple calls. Setting `session.headers.update({...})` merges your headers into every request made through `session.get(...)`, `session.post(...)`, and so on — you never repeat them per-call again.

There's a real efficiency reason too, not just cleanliness: a `Session` reuses the underlying TCP connection across calls to the same host, instead of opening a fresh one every time — genuinely faster for multiple calls to the same API.

The exact pieces:

- `requests.Session()` — construct once, reuse everywhere; don't create a new one per call, that defeats the whole point.
- `session.headers.update(dict)` — merges into the session's persistent headers dict; call it once, right after creating the session.
- `session.get(url)` / `session.post(url, ...)` — identical signatures to the module-level `requests.get` / `requests.post`, just bound to the session's state.
- **Confirming it actually worked** — inspect `session.headers["Authorization"]` after setting it, and/or check `response.request.headers` on one of the responses you get back, to see the header that was actually sent.

Sketch how you'd rewrite 3 separate `requests.get(url, headers=headers)` calls using one shared session, before checking Hint 2.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about who else might use this session besides the code you're writing right now. If a `Session` object gets shared across threads — say, a web server handling multiple requests at once, each one calling out to the same API — mutating `session.headers` from one thread while another thread is mid-request can cause subtle, hard-to-reproduce bugs, because `Session` was never designed to have its state changed concurrently. The safe pattern is: configure a session's headers once, right after creating it, and treat it as read-only after that — never mutate a shared session's headers per-request from different call sites.

There's also a resource-cleanup question a quick script doesn't need to think about, but a long-running program does: a `Session` holds open connections in a pool. `with requests.Session()` as `session:` closes them properly when you're done, the same way `with open(path) as f:` closes a file — worth doing for a session that has a clear "done with this" point, though a session meant to live for your whole program's lifetime (like one created once in `http_client.py`) usually just lives until the process exits instead.

The real design question isn't "how do I set a header once" — it's "who else in this program might touch this session, and could two different places disagree about what's in it?" A single-purpose session built once at startup and never touched again is safe. A session whose headers get changed mid-program, from more than one place, is a bug waiting for the right timing to trigger it.

The extra pieces:

- **Treat a shared session's headers as set-once, read-only** — configure right after creation, never mutate later from a different function or thread.
- `with requests.Session() as session:` — closes pooled connections explicitly when a session's lifetime has a clear end.
- **Per-request headers, when you genuinely need one call to differ** — `session.get(url, headers={"X-Extra": "..."})` merges just for that one call, without touching the session's persistent headers at all; this is the safe way to vary one call without risking the mutate-a-shared-session problem above.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both show how to set a session's headers once and reuse it correctly for a single-threaded script. Advanced asks what happens once more than one part of a program — possibly running concurrently — shares that same session object: mutating shared state from multiple places is a real, common source of bugs, and the fix (configure once at creation, use per-call `headers=` for anything that varies, close explicitly when a session's lifetime has an end) is exactly the kind of thing that doesn't show up until this code is reused somewhere bigger than the exercise.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make a session
set its Authorization header once

call session.get 2-3 times against different URLs
print the status code each time

confirm: print session.headers, check "Authorization" is there
```

Here is almost the whole thing:

```
import requests

session = requests.Session()
session.headers.update({"Authorization": "Bearer fake-token"})

response = session.get("https://api.github.com")
print(response.status_code)
```

Expected output if you run just this: 200 . What's missing: 2 more calls through the same session, and printing `session.headers` to confirm the header sticks. Add those yourself, then check the Intermediate Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
session = requests.Session()
session.headers.update({"Authorization": "Bearer fake-token"})

for url in [url1, url2, url3]:
    response = session.get(url)
    print(url, "->", response.status_code)

print(session.headers["Authorization"])
```

```
import requests

session = requests.Session()
session.headers.update({"Authorization": "Bearer fake-token"})

urls = ["https://api.github.com", "https://api.github.com/zen"]

for url in urls:
    response = session.get(url)
    print(f"{url} -> {response.status_code}")
```

What's missing: confirming the header was actually *sent*, not just set. Write that check yourself — look at `response.request.headers` on one of the responses — then compare against the [Solution](#). Notice nothing about the loop needs to know or care about the header — that's the entire point of the session: the calling code gets simpler, not more complicated, as you add more calls.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
class ApiClient:
    holds: base_url, a session (configured once in __init__)

    method get(path):
        return self.session.get(self.base_url + path)

make one ApiClient
call .get(...) a few times through it, with different paths
never touch client.session.headers again after __init__
```

Here's almost the whole thing — fill in the missing piece yourself:

```
import requests

class ApiClient:
    def __init__(self, base_url: str, token: str) -> None:
        self.base_url = base_url
        self.session = requests.Session()
        self.session.headers.update({"Authorization": f"Bearer {token}"})

    def get(self, path: str) -> requests.Response:
        # your turn: call self.session.get on self.base_url + path
        ...

client = ApiClient(base_url="https://api.github.com", token="fake-token")
for path in ["", "/zen", "/octocat"]:
    response = client.get(path)
    print(f"{path or '/'} -> {response.status_code}")
```

Fill in `get()` yourself, then compare all 3 of your finished versions against the [Solution](#) — its Advanced version also shows the `with` form and explains exactly when the shared-session mutation problem actually bites.

Difference between Basic, Intermediate, and Advanced: same underlying idea (set headers once, reuse the session) at 3 completeness levels — Basic and Intermediate prove it works for a straight-line script. Advanced wraps the session inside a small class that owns it and never exposes

a reason to mutate its headers again after construction, which is the actual shape you want once this code is imported and called from more than one place in a bigger program, as it will be in the Build Task.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)